(19) VISITOR PATTERN :- SINGLE | DOUBLE DISPATCH

-> Behavioral pattern allows us to add new operation to existing class without changing their structure.

-> Achieved by seperating operations from the object / class

-> also achieves double dispatch.

---

class PROBLEM

Room
- getPrice()
- getAvailability()
- getMaintainence()

-> when new features / operation gets added, class will grow heavily.

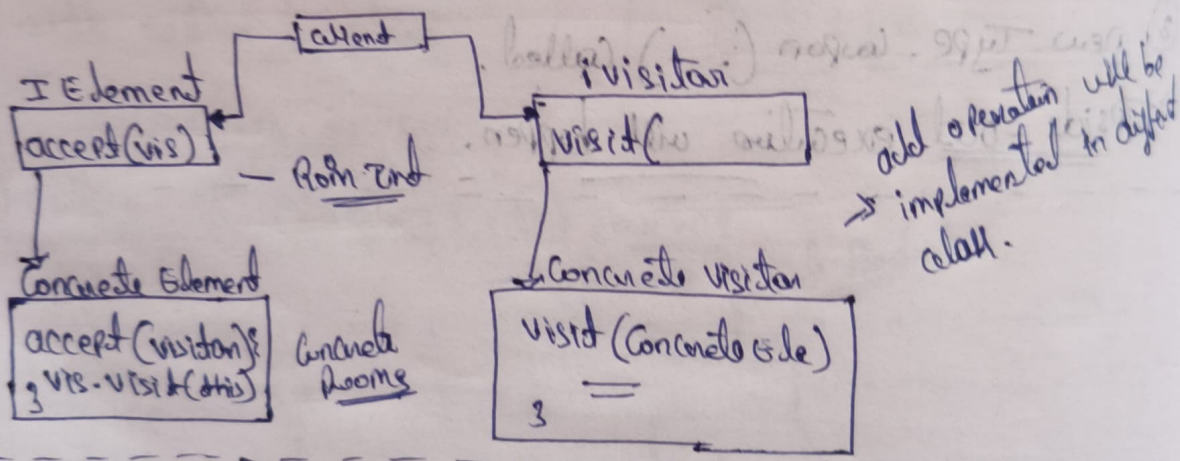
-> To avoid this, we need to use visitor

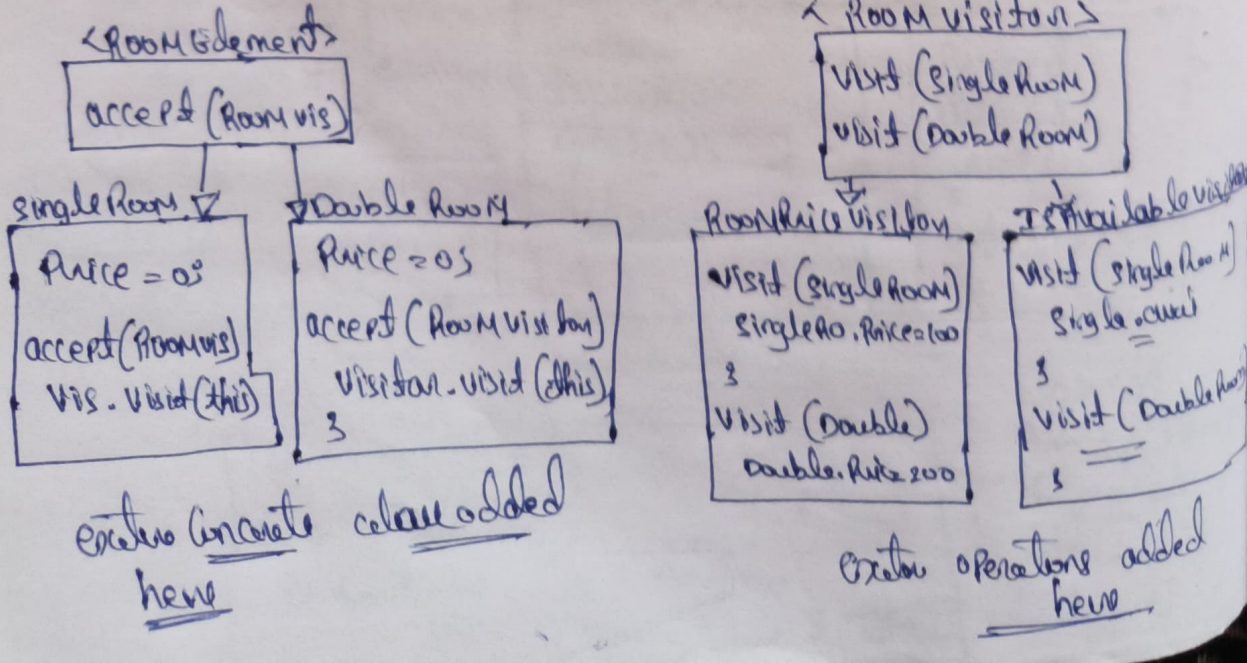
-> need to test the class again, when new operation is added.

SOLUTION

① Make class has Element interface

② Create visitor interface & add all method / operation under





Main()
RoomEld Single = new SingleRoom ();
RoomEde Double = new DoubleRoom ();

RoomVisitor Price = new RoomPriceVis ();
single. accept (Price);
single. accept (new IsAvailable ());

→ SINGLE DISPATCH :- when we call any method using single Reference object, then it is single dispatch.

SingleRoom room = new SingleRoom()
room.accept() Single Ref → room

→ Double Dispatch :- Now we are using double dispatch method will be Invoked by one object & another Reference object is passed as argument

single.accept (Price); single = 1st Reference
Price = 2nd Reference → passed as argument